

P. Sasi Kumar

Data Analytics

Level 1

Task 3 : Cleaning Data [amazon_sales_data_uncleaned]

```
# STEP 1: Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# STEP 2: Load the dataset
FILE_PATH = "/content/amazon_sales_data_uncleaned.csv"

df = pd.read_csv(FILE_PATH)

print("Dataset loaded successfully.")
print("Shape:", df.shape)

Dataset loaded successfully.
Shape: (8127, 16)

# STEP 3: Initial inspection
print("\nFirst 5 rows:")
display(df.head())

print("\nDataset information:")
df.info()

print("\nStatistical summary:")
display(df.describe(include="all").T)
```

First 5 rows:

```
{"summary": "{\n  \"name\": \"display(df\", \n  \"rows\": 5, \n  \"fields\": [\n    {\n      \"column\": \"Title\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 5, \n        \"samples\": [\n          \"LISEN USB C to Lightning Cable, 240W 4 in 1 Charging Cable 6.6FT, Chubby USB A/C to C/Lightning with Light for iPhone 16e 15 14 Pro/MacBook Air 17/iPad/Samsung/Switch 2, Multi Chargers for All Devices\", \n          \"Apple AirTag 4 Pack. Keep Track of and find Your Keys, Wallet, Luggage, Backpack, and More. Simple one-tap Set up with iPhone or iPad\", \n          \"DJI Mic 2 (2 TX + 1 RX + Charging Case), Wireless Lavalier Microphone, Intelligent Noise Cancelling, 32-bit Float Internal Recording, 820 ft.(250m) Range, Microphone for iPhone, Android, Camera\", \n          ], \n          \"semantic_type\": \"\", \n          \"\"}
```

```

\"description\": \"\\n      }\n    },\n    {\n      \"column\":
\"Rating\", \n      \"properties\": {\n        \"dtype\": \"string\", \n
\"num_unique_values\": 3, \n        \"samples\": [\n          \"4.6 out
of 5 stars\", \n          \"4.3 out of 5 stars\", \n          \"4.8 out
of 5 stars\" \n        ], \n        \"semantic_type\": \"\\n\", \n
\"description\": \"\\n      }\n    },\n    {\n      \"column\":
\"Number_of_reviews\", \n      \"properties\": {\n        \"dtype\":
\"string\", \n        \"num_unique_values\": 5, \n        \"samples\":
[\n          \"2,457\", \n          \"28,988\", \n          \"3,044\" \n
        ], \n        \"semantic_type\": \"\\n\", \n        \"description\": \"\\n
      }\n    },\n    {\n      \"column\": \"bought_in_last_month\", \n
      \"properties\": {\n        \"dtype\": \"string\", \n
        \"num_unique_values\": 4, \n        \"samples\": [\n          \"6K+
bought in past month\", \n          \"10K+ bought in past month\", \n
          \"300+ bought in past month\" \n        ], \n        \"semantic_type\":
\"\\n\", \n        \"description\": \"\\n      }\n    },\n    {\n      \"column\": \"Current/discounted_price\", \n
      \"properties\": {\n        \"dtype\": \"string\", \n
        \"num_unique_values\": 3, \n        \"samples\": [\n          \"89.68\", \n
          \"9.99\", \n          \"314\" \n        ], \n        \"semantic_type\": \"\\n\", \n
        \"description\": \"\\n      }\n    },\n    {\n      \"column\":
\"Price_on_variant\", \n      \"properties\": {\n        \"dtype\":
\"string\", \n        \"num_unique_values\": 4, \n        \"samples\":
[\n          \"basic variant price: nan\", \n          \"basic variant
price: $72.74\", \n          \"basic variant price: 2.4GHz\" \n
        ], \n        \"semantic_type\": \"\\n\", \n        \"description\": \"\\n
      }\n    },\n    {\n      \"column\": \"Listed_price\", \n
      \"properties\": {\n        \"dtype\": \"string\", \n
        \"num_unique_values\": 4, \n        \"samples\":
[\n          \"$15.99\", \n          \"No Discount\", \n          \"$159.00\" \n
        ], \n        \"semantic_type\": \"\\n\", \n        \"description\": \"\\n
      }\n    },\n    {\n      \"column\": \"is_best_seller\", \n
      \"properties\": {\n        \"dtype\": \"category\", \n
        \"num_unique_values\": 2, \n        \"samples\":
[\n          \"Best Seller\", \n          \"No Badge\" \n        ], \n        \"semantic_type\": \"\\n\", \n
        \"description\": \"\\n      }\n    },\n    {\n      \"column\": \"is_sponsored\", \n
      \"properties\": {\n        \"dtype\": \"category\", \n
        \"num_unique_values\": 2, \n        \"samples\":
[\n          \"Organic\", \n          \"Sponsored\" \n        ], \n        \"semantic_type\": \"\\n\", \n
        \"description\": \"\\n      }\n    },\n    {\n      \"column\": \"is_couponed\", \n
      \"properties\": {\n        \"dtype\": \"category\", \n
        \"num_unique_values\": 2, \n        \"samples\":
[\n          \"No Coupon\", \n          \"Save 15% with coupon\" \n        ], \n        \"semantic_type\": \"\\n\", \n
        \"description\": \"\\n      }\n    },\n    {\n      \"column\": \"buy_box_availability\", \n
      \"properties\": {\n        \"dtype\": \"category\", \n
        \"num_unique_values\": 1, \n        \"samples\": [\n          \"Add to

```

```
\n\n    \n        \"semantic_type\": \"\", \n        \"description\": \"\", \n        \"column\": \"\", \n        \"properties\": { \n            \"dtype\": \"category\", \n            \"num_unique_values\": 2, \n            \"samples\": [ \n                \"Delivery Fri, Aug 29\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\", \n            \"column\": \"sustainability_badges\", \n            \"properties\": { \n                \"dtype\": \"category\", \n                \"num_unique_values\": 1, \n                \"samples\": [ \n                    \"Carbon impact\", \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\", \n                \"column\": \"\", \n                \"image_url\": \"\", \n                \"properties\": { \n                    \"dtype\": \"string\", \n                    \"string\": \"\", \n                    \"num_unique_values\": 5, \n                    \"samples\": [ \n                        \"https://m.media-amazon.com/images/I/61nbF6aVIPL._AC_UL320_.jpg\", \n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\", \n                    \"column\": \"product_url\", \n                    \"properties\": { \n                        \"dtype\": \"string\", \n                        \"num_unique_values\": 5, \n                        \"samples\": [ \n                            \"/sspa/click?ie=UTF8&spc=MT04NzEzNDY2NTQ5NDYxNDQ20jE3NTU4MDAwNjg6c3BfYXRmX2Jyb3dzZT0zMDA4NjA2Mzc2NDIxMDI0jA60g&url=%2FLISEN-Charging-Lighting-MacBook-Chargers%2Fdpc%2FB0DHWZCZDZ%2Fref%3Ds_r_1_2_sspa%3Fdb%3DeyJ2IjojMSJ9.avmZLHCQuV0wikquBqYSIJN8SVcyxtkXHQMPT7Zjzkf4TeZrZfQETMhdWuWgtTrVz8ITKpXLHvZj0fZRjxqqMPYNMitRqeUoeIwdYfc5nnzJ8m0T8HYeedlh3YSOhJQjeHskevMUQWyg6TtoB2tHcHtedYPsQ6VwFQTl6avsPrgVpFKrto3ff9TDR9BcRyPwM6AiYnvh7wA5PP9DjZddhCPf7bVPcHMZ6Hwd40dQDWm9_M8R-LcwKY8wnuWRUSplhfYJBVTjAgSb3Y3y88VGpfqY3V8Rd2ge-woBzUMA.yqwi-krmElGgXIMa8kKUPx1XmaXd9lHaAkMkVVpStxo%26dib_tag%3Dse%26qid%3D1755800068%26refinements%3Dp_n_g-101014971069111%253A119653281011%26s%3Delectronics%26sr%3D1-2-spons%26sp_csd%3Dd2lkZ2V0TmFtZT1zcF9hdGZfYnJvd3Nl%26psc%3D1\", \n                    ], \n                    \"semantic_type\": \"\", \n                    \"description\": \"\", \n                    \"column\": \"collected_at\", \n                    \"properties\": { \n                        \"dtype\": \"object\", \n                        \"num_unique_values\": 1, \n                        \"samples\": [ \n                            \"21-08-2025 11:14\", \n                        ], \n                        \"semantic_type\": \"\", \n                        \"description\": \"\", \n                    }, \n                }, \n            }, \n        }, \n    }, \n    {\n        \"column\": \"product_url\", \n        \"properties\": {\n            \"dtype\": \"string\", \n            \"num_unique_values\": 5, \n            \"samples\": [\n                \"/sspa/click?ie=UTF8&spc=MT04NzEzNDY2NTQ5NDYxNDQ20jE3NTU4MDAwNjg6c3BfYXRmX2Jyb3dzZT0zMDA4NjA2Mzc2NDIxMDI0jA60g&url=%2FLISEN-Charging-Lighting-MacBook-Chargers%2Fdpc%2FB0DHWZCZDZ%2Fref%3Ds_r_1_2_sspa%3Fdb%3DeyJ2IjojMSJ9.avmZLHCQuV0wikquBqYSIJN8SVcyxtkXHQMPT7Zjzkf4TeZrZfQETMhdWuWgtTrVz8ITKpXLHvZj0fZRjxqqMPYNMitRqeUoeIwdYfc5nnzJ8m0T8HYeedlh3YSOhJQjeHskevMUQWyg6TtoB2tHcHtedYPsQ6VwFQTl6avsPrgVpFKrto3ff9TDR9BcRyPwM6AiYnvh7wA5PP9DjZddhCPf7bVPcHMZ6Hwd40dQDWm9_M8R-LcwKY8wnuWRUSplhfYJBVTjAgSb3Y3y88VGpfqY3V8Rd2ge-woBzUMA.yqwi-krmElGgXIMa8kKUPx1XmaXd9lHaAkMkVVpStxo%26dib_tag%3Dse%26qid%3D1755800068%26refinements%3Dp_n_g-101014971069111%253A119653281011%26s%3Delectronics%26sr%3D1-2-spons%26sp_csd%3Dd2lkZ2V0TmFtZT1zcF9hdGZfYnJvd3Nl%26psc%3D1\", \n            ], \n            \"semantic_type\": \"\", \n            \"description\": \"\", \n            \"column\": \"collected_at\", \n            \"properties\": {\n                \"dtype\": \"object\", \n                \"num_unique_values\": 1, \n                \"samples\": [\n                    \"21-08-2025 11:14\", \n                ], \n                \"semantic_type\": \"\", \n                \"description\": \"\", \n            }, \n        }, \n    }, \n], \n\"type\": \"dataframe\"}
```

Dataset information:

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 8127 entries, 0 to 8126
```

Data columns (total 16 columns):

#	Column	Non-Null Count	Dtype
0	Title	8127 non-null	object
1	Rating	8112 non-null	object
2	Number of reviews	8112 non-null	object

3	bought_in_last_month	7410	non-null	object
4	Current/discounted_price	6053	non-null	object
5	Price_on_variant	8127	non-null	object
6	Listed_price	8127	non-null	object
7	is_best_seller	8127	non-null	object
8	is_sponsored	8127	non-null	object
9	is_couponed	8127	non-null	object
10	buy_box_availability	5617	non-null	object
11	delivery_details	6046	non-null	object
12	sustainability_badges	910	non-null	object
13	image_url	8127	non-null	object
14	product_url	7564	non-null	object
15	collected_at	8126	non-null	object

dtypes: object(16)

memory usage: 1016.0+ KB

Statistical summary:

```
{
  "summary": {
    "name": "display(df",
    "rows": 16,
    "fields": [
      {
        "column": "count",
        "properties": {
          "dtype": "date",
          "min": "910",
          "max": "8127",
          "num_unique_values": 9,
          "samples": [
            "7564",
            "8112",
            "6046"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "unique",
        "properties": {
          "dtype": "date",
          "min": 1,
          "max": 7564,
          "num_unique_values": 16,
          "samples": [
            6315,
            25,
            2687
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "top",
        "properties": {
          "dtype": "string",
          "num_unique_values": 16,
          "samples": [
            "Sanus Height Adjustable Speaker Stand for Sonos Era 300\\u2122 - 17\\\" Height Adjustment Sonos Stand Includes Carpet Spikes & Rubber Pads - Easy DIY Install Comes w/All Hardware - White",
            "4.7 out of 5 stars",
            "basic variant price: nan"
          ],
          "semantic_type": "",
          "description": ""
        }
      },
      {
        "column": "freq",
        "properties": {
          "dtype": "date",
          "min": "1",
          "max": "7784",
          "num_unique_values": 16,
          "samples": [
            "206",
            "1337",
            "3743"
          ],
          "semantic_type": "",
          "description": ""
        }
      }
    ],
    "type": "dataframe"
  }
}
```

STEP 4: Create initial data-quality report

before_rows = len(df)

before_columns = len(df.columns)

before_duplicates = df.duplicated().sum()

quality_before = pd.DataFrame({

```

    "Column": df.columns,
    "Data_Type": df.dtypes.astype(str).values,
    "Missing_Values": df.isnull().sum().values,
    "Missing_%": (df.isnull().mean() * 100).round(2).values,
    "Unique_Values": df.nunique().values
})

print("\n===== DATA QUALITY REPORT BEFORE CLEANING =====")
display(quality_before)

```

```
===== DATA QUALITY REPORT BEFORE CLEANING =====
```

```

{"summary": "{\n  \"name\": \"quality_before\",\n  \"rows\": 16,\n  \"fields\": [\n    {\n      \"column\": \"Column\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 16,\n        \"samples\": [\n          \"Title\",\n          \"Rating\",\n          \"Price_on_variant\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Data_Type\",\n      \"properties\": {\n        \"dtype\": \"category\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"object\"\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Missing_Values\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 1887,\n        \"min\": 0,\n        \"max\": 7217,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          563\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Missing_%\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 23.22084199413679,\n        \"min\": 0.0,\n        \"max\": 88.8,\n        \"num_unique_values\": 9,\n        \"samples\": [\n          6.93\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    },\n    {\n      \"column\": \"Unique_Values\",\n      \"properties\": {\n        \"dtype\": \"number\",\n        \"std\": 2622,\n        \"min\": 1,\n        \"max\": 7564,\n        \"num_unique_values\": 16,\n        \"samples\": [\n          6315\n        ],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n      }\n    }\n  ],\n  \"type\": \"dataframe\",\n  \"variable_name\": \"quality_before\"}

```

```
# STEP 5: Check duplicate records
```

```
print("\nDuplicate rows:", before_duplicates)
```

```

if before_duplicates > 0:
    print("\nSample duplicate rows:")
    display(df[df.duplicated()].head())

```

```
Duplicate rows: 272
```

Sample duplicate rows:

```
{"repr_error": "0", "type": "dataframe"}  
  
# STEP 6: Check missing values  
missing_before = df.isnull().sum()  
  
print("\nMissing values before cleaning:")  
display(missing_before[missing_before > 0])
```

Missing values before cleaning:

Rating	15
Number_of_reviews	15
bought_in_last_month	717
Current/discounted_price	2074
buy_box_availability	2510
delivery_details	2081
sustainability_badges	7217
product_url	563
collected_at	1

dtype: int64

```
# STEP 7: Standardize column names  
df.columns = (  
    df.columns  
    .str.strip()  
    .str.lower()  
    .str.replace(" ", "_")  
    .str.replace(r"^\w+", "_", regex=True)  
    .str.strip("_")  
)  
  
print("\nStandardized column names:")  
print(df.columns.tolist())
```

Standardized column names:

```
['title', 'rating', 'number_of_reviews', 'bought_in_last_month',  
'current_discounted_price', 'price_on_variant', 'listed_price',  
'is_best_seller', 'is_sponsored', 'is_couponed',  
'buy_box_availability', 'delivery_details', 'sustainability_badges',  
'image_url', 'product_url', 'collected_at']
```

```
# STEP 8: Standardize categorical/text values  
# Remove unnecessary spaces and standardize text formatting  
for col in df.select_dtypes(include="object").columns:  
    df[col] = df[col].astype("string").str.strip()
```

```

# Convert common gender representations if a gender column exists
if "gender" in df.columns:
    df["gender"] = (
        df["gender"]
        .str.lower()
        .replace({
            "m": "Male",
            "male": "Male",
            "man": "Male",
            "f": "Female",
            "female": "Female",
            "woman": "Female"
        })
    )

# STEP 9: Convert date columns
# Automatically detect columns containing 'date'
date_columns = [col for col in df.columns if "date" in col]

for col in date_columns:
    df[col] = pd.to_datetime(df[col], errors="coerce")

print("\nDate columns converted:")
print(date_columns)

Date columns converted:
[]

# STEP 10: Convert numeric columns
# Automatically convert columns that contain mostly numeric values
for col in df.columns:
    if df[col].dtype == "object" or str(df[col].dtype) == "string":

        cleaned_values = (
            df[col]
            .astype("string")
            .str.replace(",", "", regex=False)
            .str.replace("$", "", regex=False)
            .str.replace("%", "", regex=False)
            .str.strip()
        )

        numeric_values = pd.to_numeric(cleaned_values,
errors="coerce")

        # Convert only when most values are numeric
        if numeric_values.notna().mean() >= 0.80:
            df[col] = numeric_values

```

```

# STEP 11: Handle missing categorical values
categorical_columns = df.select_dtypes(
    include=["object", "string", "category"]
).columns

for col in categorical_columns:
    if df[col].isnull().sum() > 0:
        mode_value = df[col].mode()

        if len(mode_value) > 0:
            df[col] = df[col].fillna(mode_value.iloc[0])

print("\nCategorical missing values handled.")

```

Categorical missing values handled.

```

# STEP 12: Handle missing numerical values
numeric_columns = df.select_dtypes(include=np.number).columns

for col in numeric_columns:
    if df[col].isnull().sum() > 0:

        # Median is more robust when outliers exist
        median_value = df[col].median()

        # If the column is an integer type but the median is a float,
        # convert the column to a float type to accommodate the
        median.
        if pd.api.types.is_integer_dtype(df[col]) and
        isinstance(median_value, float):
            df[col] = df[col].astype('Float64') # Using pandas'
            nullable float type

            df[col] = df[col].fillna(median_value)

print("Numerical missing values handled using median imputation.")

```

Numerical missing values handled using median imputation.

```

# STEP 13: Detect numerical outliers using IQR
outlier_summary = []

for col in numeric_columns:

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

```

```

outliers = df[
    (df[col] < lower_bound) |
    (df[col] > upper_bound)
]

outlier_summary.append({
    "Column": col,
    "Q1": Q1,
    "Q3": Q3,
    "Lower_Bound": lower_bound,
    "Upper_Bound": upper_bound,
    "Outlier_Count": len(outliers)
})

outlier_report = pd.DataFrame(outlier_summary)

print("\n===== OUTLIER REPORT =====")
display(outlier_report)

```

```
===== OUTLIER REPORT =====
```

```

{"summary": "{\n  \"name\": \"outlier_report\",\n  \"rows\": 1,\n  \"fields\": [\n    {\n      \"column\": \"Column\",\n      \"properties\": {\n        \"dtype\": \"string\",\n        \"num_unique_values\": 1,\n        \"samples\": [\n          \"number_of_reviews\",\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        ],\n        \"column\": \"Q1\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": null,\n          \"min\": 180.0,\n          \"max\": 180.0,\n          \"num_unique_values\": 1,\n          \"samples\": [\n            180.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        \"column\": \"Q3\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": null,\n          \"min\": 5061.0,\n          \"max\": 5061.0,\n          \"num_unique_values\": 1,\n          \"samples\": [\n            5061.0\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        \"column\": \"Lower_Bound\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": null,\n          \"min\": -7141.5,\n          \"max\": -7141.5,\n          \"num_unique_values\": 1,\n          \"samples\": [\n            -7141.5\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        \"column\": \"Upper_Bound\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\": null,\n          \"min\": 12382.5,\n          \"max\": 12382.5,\n          \"num_unique_values\": 1,\n          \"samples\": [\n            12382.5\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        },\n        \"column\": \"Outlier_Count\",\n        \"properties\": {\n          \"dtype\": \"number\",\n          \"std\":

```

```

null,\n          \"min\": 838,\n          \"max\": 838,\n          \"num_unique_values\": 1,\n          \"samples\": [\n            838\n          ],\n          \"semantic_type\": \"\",\n          \"description\": \"\"\n        }\n      }\n    ]\n  }\", \"type\": \"dataframe\", \"variable_name\": \"outlier_report\"}

```

```

# STEP 14: Handle outliers using IQR capping
# We cap extreme values instead of deleting rows.
for col in numeric_columns:

```

```

    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1

```

```

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

```

```

    df[col] = df[col].clip(
        lower=lower_bound,
        upper=upper_bound
    )

```

```

print("\nOutliers handled using IQR capping.")

```

Outliers handled using IQR capping.

```

# STEP 15: Remove duplicate rows
duplicates_removed = df.duplicated().sum()

```

```

df = df.drop_duplicates().reset_index(drop=True)

```

```

print("\nDuplicate rows removed:", duplicates_removed)
print("New dataset shape:", df.shape)

```

Duplicate rows removed: 272
New dataset shape: (7855, 16)

```

# STEP 16: Create BEFORE vs AFTER quality report
after_rows = len(df)
after_duplicates = df.duplicated().sum()

```

```

quality_after = pd.DataFrame({
    "Column": df.columns,
    "Data_Type": df.dtypes.astype(str).values,
    "Missing_Values": df.isnull().sum().values,
    "Missing_%": (df.isnull().mean() * 100).round(2).values,
    "Unique_Values": df.nunique().values
})

```

```

comparison = pd.DataFrame({

```

```

    "Metric": [
        "Rows",
        "Columns",
        "Duplicate Rows",
        "Total Missing Values"
    ],
    "Before": [
        before_rows,
        before_columns,
        before_duplicates,
        missing_before.sum()
    ],
    "After": [
        after_rows,
        len(df.columns),
        after_duplicates,
        df.isnull().sum().sum()
    ]
}
})

print("\n===== BEFORE VS AFTER =====")
display(comparison)

print("\n===== FINAL DATA QUALITY REPORT =====")
display(quality_after)

```

===== BEFORE VS AFTER =====

```

{"summary":{"\n  \"name\": \"comparison\", \n  \"rows\": 4, \n  \"fields\": [\n    {\n      \"column\": \"Metric\", \n      \"properties\": {\n        \"dtype\": \"string\", \n        \"num_unique_values\": 4, \n        \"samples\": [\n          \"Columns\", \n          \"Total Missing Values\", \n          \"Rows\" \n        ], \n        \"semantic_type\": \"\", \n        \"description\": \"\" \n      }, \n      {\n        \"column\": \"Before\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 7248, \n          \"min\": 16, \n          \"max\": 15193, \n          \"num_unique_values\": 4, \n          \"samples\": [\n            16, \n            15193, \n            8127 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        }, \n      {\n        \"column\": \"After\", \n        \"properties\": {\n          \"dtype\": \"number\", \n          \"std\": 3924, \n          \"min\": 0, \n          \"max\": 7855, \n          \"num_unique_values\": 3, \n          \"samples\": [\n            7855, \n            16, \n            0 \n          ], \n          \"semantic_type\": \"\", \n          \"description\": \"\" \n        } \n      ] \n    } \n  ], \"type\": \"dataframe\", \"variable_name\": \"comparison\"}

```

===== FINAL DATA QUALITY REPORT =====

```
{
  "summary": {
    "name": "quality_after",
    "rows": 16,
    "fields": [
      {
        "column": "Column",
        "properties": {
          "dtype": "string",
          "num_unique_values": 16,
          "samples": [
            "title",
            "rating",
            "price_on_variant",
            "semantic_type": "",
            "description": ""
          ],
          "column": "Data_Type",
          "properties": {
            "dtype": "category",
            "num_unique_values": 2,
            "samples": [
              "Float64",
              "string"
            ],
            "semantic_type": "",
            "description": ""
          ],
          "column": "Missing_Values",
          "properties": {
            "dtype": "number",
            "std": 0,
            "min": 0,
            "max": 0,
            "num_unique_values": 1,
            "samples": [0],
            "semantic_type": "",
            "description": ""
          ],
          "column": "Missing_%",
          "properties": {
            "dtype": "number",
            "std": 0.0,
            "min": 0.0,
            "max": 0.0,
            "num_unique_values": 1,
            "samples": [0.0],
            "semantic_type": "",
            "description": ""
          ],
          "column": "Unique_Values",
          "properties": {
            "dtype": "number",
            "std": 2589,
            "min": 1,
            "max": 7564,
            "num_unique_values": 16,
            "samples": [6315],
            "semantic_type": "",
            "description": ""
          ]
        ]
      }
    ],
    "type": "dataframe",
    "variable_name": "quality_after"
  }
}
```

STEP 17: Visualize missing values

```
missing_comparison = pd.DataFrame({
    "Before": missing_before.sum(),
    "After": df.isnull().sum()
})

missing_comparison = missing_comparison[
    (missing_comparison["Before"] > 0) |
    (missing_comparison["After"] > 0)
]

if not missing_comparison.empty:
    missing_comparison.plot(
        kind="bar",
        figsize=(12, 5)
    )

    plt.title("Missing Values: Before vs After Cleaning")
    plt.xlabel("Columns")
    plt.ylabel("Number of Missing Values")
    plt.xticks(rotation=45)
```

```

plt.tight_layout()
plt.show()

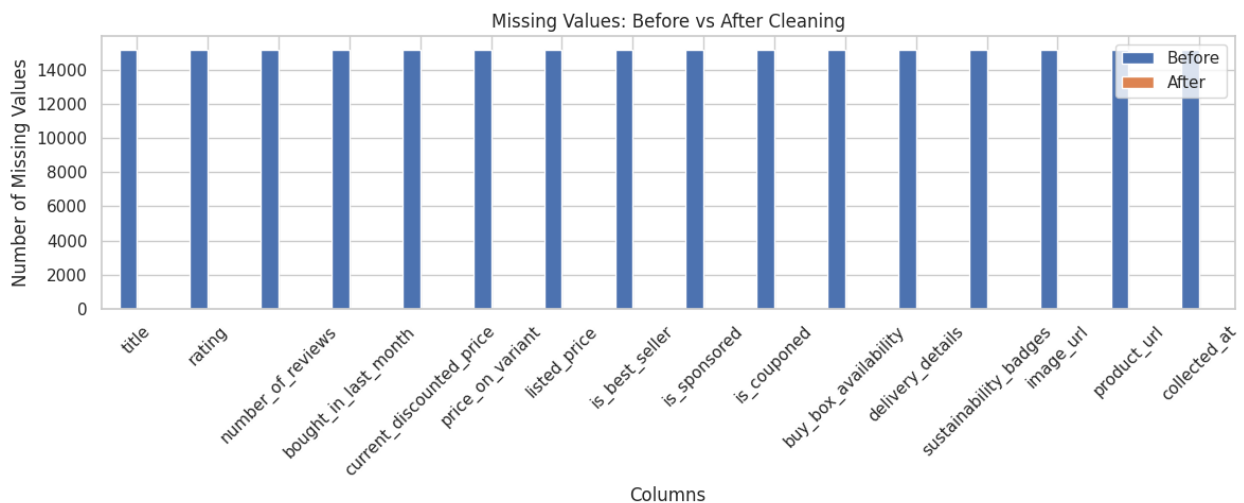
# Additional useful visualization: correlation heatmap
if len(numeric_columns) >= 2:

    plt.figure(figsize=(10, 7))

    sns.heatmap(
        df[numeric_columns].corr(),
        annot=True,
        cmap="coolwarm",
        fmt=".2f"
    )

    plt.title("Correlation Between Numerical Variables")
    plt.tight_layout()
    plt.show()

```



```

# STEP 19: Generate useful insights
print("\n===== KEY DATA QUALITY INSIGHTS =====")

print(f"Original rows: {before_rows}")
print(f"Final rows: {after_rows}")
print(f"Duplicate rows removed: {duplicates_removed}")
print(f"Missing values before cleaning: {missing_before.sum()}")
print(f"Missing values after cleaning: {df.isnull().sum().sum()}")

if len(numeric_columns) > 0:

    print("\nNumerical variable summary:")

    for col in numeric_columns:

```

```

        print(
            f"{col}: "
            f"Min={df[col].min():.2f}, "
            f"Median={df[col].median():.2f}, "
            f"Max={df[col].max():.2f}"
        )

print("\nData cleaning completed successfully.")

```

===== KEY DATA QUALITY INSIGHTS =====

Original rows: 8127

Final rows: 7855

Duplicate rows removed: 272

Missing values before cleaning: 15193

Missing values after cleaning: 0

Numerical variable summary:

number_of_reviews: Min=1.00, Median=1059.50, Max=12382.50

Data cleaning completed successfully.

Useful visualizations

Ensure 'rating' is numerical for better visualization

Extract numerical part from 'rating' column

```

if 'rating' in df.columns and df['rating'].dtype == 'string':
    df['numeric_rating'] = df['rating'].str.extract('([\d\.]
+)'').astype(float)
    df['numeric_rating'] =

```

```

df['numeric_rating'].fillna(df['numeric_rating'].median()) # fill
missing if any after extraction

```

else:

*# If 'rating' column is not a string type, assume it's already
numeric or handle as appropriate*

*# For this dataset, we expect 'rating' to be an object (string)
initially.*

*# If it's already numeric, no extraction needed. If missing,
numeric_rating will remain None.*

```

if 'rating' in df.columns and

```

```

pd.api.types.is_numeric_dtype(df['rating']):
    df['numeric_rating'] = df['rating']

```

else:

```

    df['numeric_rating'] = None # Fallback if 'rating' column is
not present or not convertible

```

1. Distribution of 'number_of_reviews'

```

if 'number_of_reviews' in df.columns and

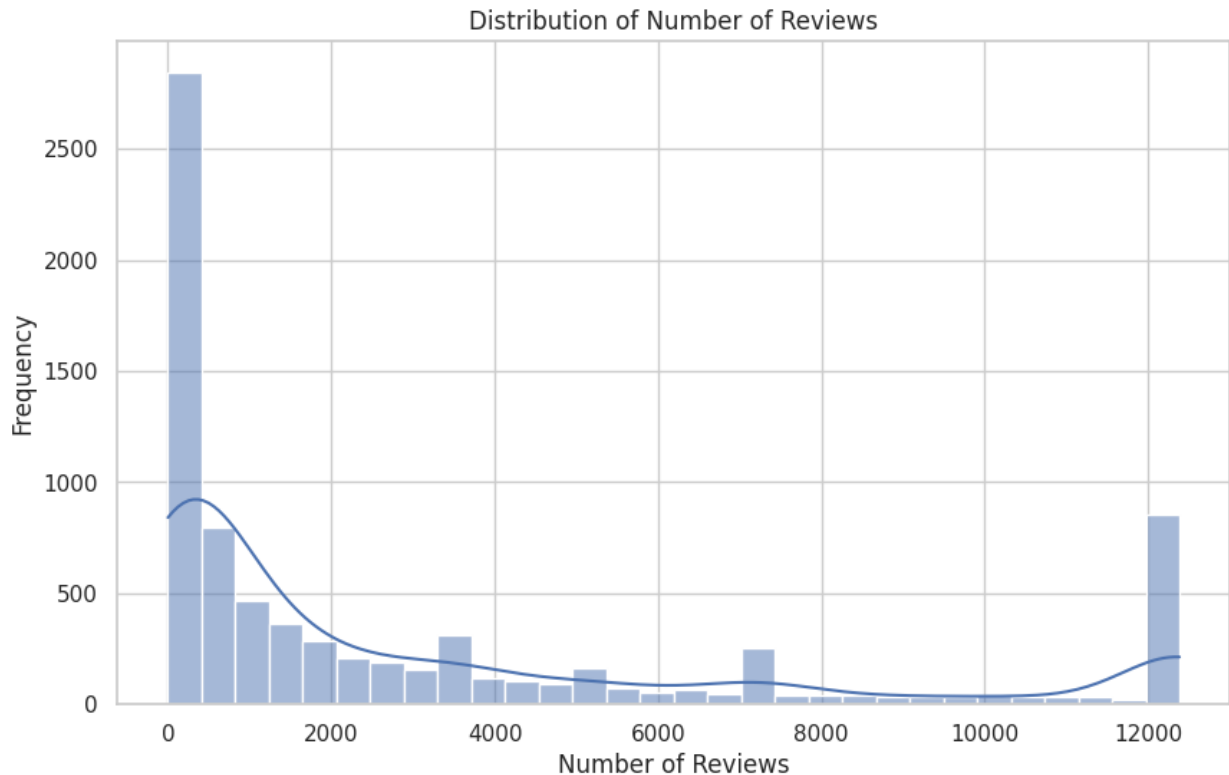
```

```

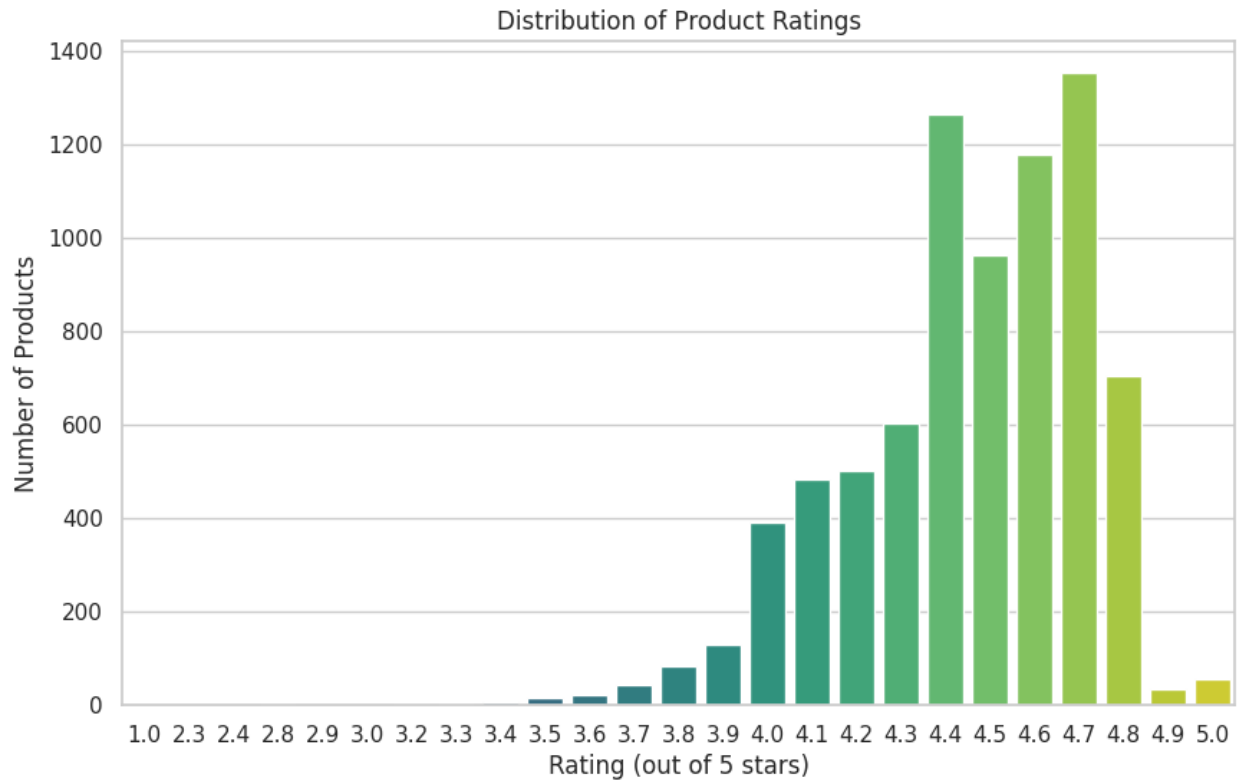
pd.api.types.is_numeric_dtype(df['number_of_reviews']):
    plt.figure(figsize=(10, 6))

```

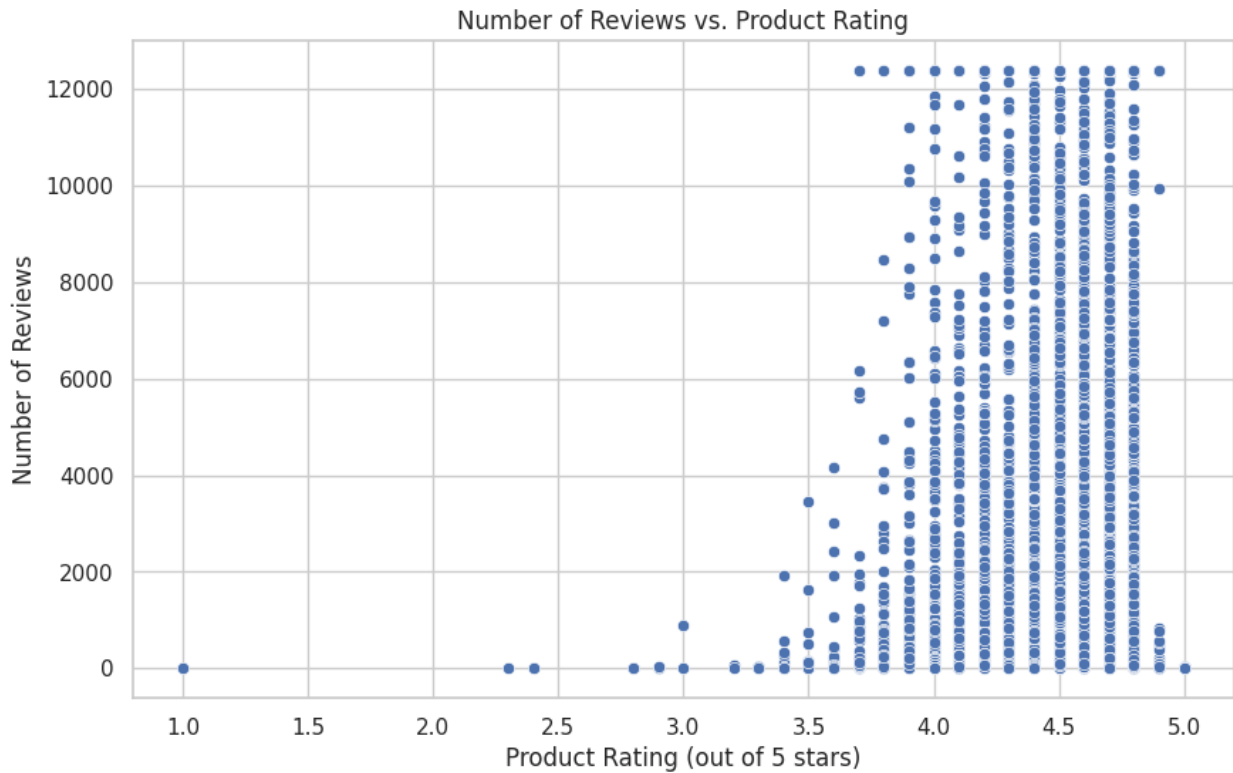
```
sns.histplot(df['number_of_reviews'], bins=30, kde=True)
plt.title('Distribution of Number of Reviews')
plt.xlabel('Number of Reviews')
plt.ylabel('Frequency')
plt.show()
```



```
# 2. Distribution of 'numeric_rating'
if df['numeric_rating'] is not None and
pd.api.types.is_numeric_dtype(df['numeric_rating']):
    plt.figure(figsize=(10, 6))
    sns.countplot(x=df['numeric_rating'].round(1).sort_values(),
palette='viridis')
    plt.title('Distribution of Product Ratings')
    plt.xlabel('Rating (out of 5 stars)')
    plt.ylabel('Number of Products')
    plt.show()
```

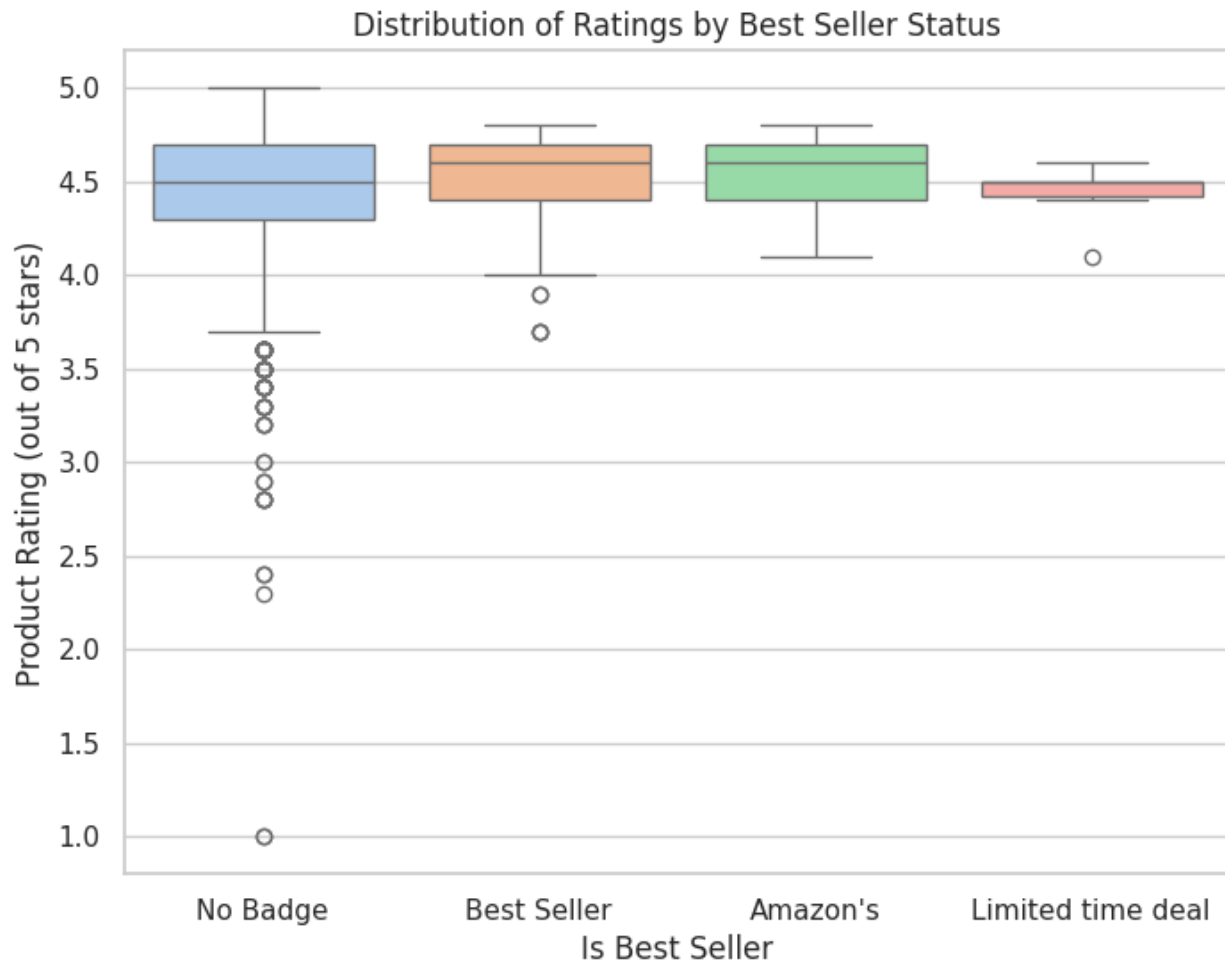


```
# 3. Scatter plot: Number of Reviews vs Rating
if 'number_of_reviews' in df.columns and df['numeric_rating'] is not
None and \
    pd.api.types.is_numeric_dtype(df['number_of_reviews']) and
    pd.api.types.is_numeric_dtype(df['numeric_rating']):
    plt.figure(figsize=(10, 6))
    sns.scatterplot(x='numeric_rating', y='number_of_reviews',
data=df)
    plt.title('Number of Reviews vs. Product Rating')
    plt.xlabel('Product Rating (out of 5 stars)')
    plt.ylabel('Number of Reviews')
    plt.show()
```



```
# Segmented analysis: Ratings by 'is_best_seller' status

if 'numeric_rating' in df.columns and 'is_best_seller' in df.columns:
    plt.figure(figsize=(8, 6))
    sns.boxplot(x='is_best_seller', y='numeric_rating', data=df,
palette='pastel')
    plt.title('Distribution of Ratings by Best Seller Status')
    plt.xlabel('Is Best Seller')
    plt.ylabel('Product Rating (out of 5 stars)')
    plt.show()
else:
    print("Cannot perform segmented analysis: 'numeric_rating' or
'is_best_seller' columns not found.")
```



```
print("\nDescriptive statistics for the cleaned dataset:")
display(df.describe(include='all').T)
```

Descriptive statistics for the cleaned dataset:

```
{
  "summary": {
    "name": "display(df",
    "rows": 17,
    "fields": [
      {
        "column": "count",
        "properties": {
          "dtype": "date",
          "min": "7855",
          "max": "7855",
          "num_unique_values": 1,
          "samples": [
            "7855"
          ]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "unique",
        "properties": {
          "dtype": "date",
          "min": 1,
          "max": 7564,
          "num_unique_values": 15,
          "samples": [
            "1"
          ]
        },
        "semantic_type": "",
        "description": ""
      },
      {
        "column": "top",
        "properties": {
          "dtype": "string",
          "num_unique_values": 15,
          "samples": [
            "Add to cart"
          ]
        },
        "semantic_type": ""
      }
    ]
  }
}
```

```

\"description\": \"\\\"\\n      }\\n    },\\n    {\\n      \"column\":
\"freq\",\\n      \"properties\": {\\n        \"dtype\": \"date\",\\n
\"min\": \"158\",\\n        \"max\": \"7855\",\\n
\"num_unique_values\": 15,\\n        \"samples\": [\\n
\"7855\"\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\\\"\\\"\\n      }\\n    },\\n    {\\n      \"column\":
\"mean\",\\n      \"properties\": {\\n        \"dtype\": \"date\",\\n
\"min\": 4.449433481858689,\\n        \"max\": 3169.6309993634627,\\n
\"num_unique_values\": 2,\\n        \"samples\": [\\n
4.449433481858689\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\\\"\\\"\\n      }\\n    },\\n    {\\n      \"column\":
\"std\",\\n      \"properties\": {\\n        \"dtype\": \"date\",\\n
\"min\": 0.28007306037665947,\\n        \"max\": 4102.142209115524,\\n
\"num_unique_values\": 2,\\n        \"samples\": [\\n
0.28007306037665947\\n      ],\\n      \"semantic_type\": \"\",\\n
\"description\": \"\\\"\\\"\\n      }\\n    },\\n    {\\n      \"column\":
\"min\",\\n      \"properties\": {\\n        \"dtype\": \"date\",\\n
\"min\": 1.0,\\n        \"max\": 1.0,\\n        \"num_unique_values\":
1,\\n        \"samples\": [\\n          1.0\\n        ],\\n
\"semantic_type\": \"\",\\n        \"description\": \"\\\"\\\"\\n      }\\n
    },\\n    {\\n      \"column\": \"25%\",\\n      \"properties\": {\\n
\"dtype\": \"date\",\\n        \"min\": 4.3,\\n        \"max\": 196.0,\\n
\"num_unique_values\": 2,\\n        \"samples\": [\\n          4.3\\n
        ],\\n        \"semantic_type\": \"\",\\n        \"description\": \"\\\"\\\"\\n
      }\\n    },\\n    {\\n      \"column\": \"50%\",\\n      \"properties\": {\\n
        \"dtype\": \"date\",\\n        \"min\": 4.5,\\n        \"max\":
1059.5,\\n        \"num_unique_values\": 2,\\n        \"samples\": [\\n
4.5\\n        ],\\n        \"semantic_type\": \"\",\\n
\"description\": \"\\\"\\\"\\n      }\\n    },\\n    {\\n      \"column\":
\"75%\",\\n      \"properties\": {\\n        \"dtype\": \"date\",\\n
\"min\": 4.7,\\n        \"max\": 4724.5,\\n
\"num_unique_values\": 2,\\n        \"samples\": [\\n          4.7\\n
        ],\\n        \"semantic_type\": \"\",\\n        \"description\": \"\\\"\\\"\\n
      }\\n    },\\n    {\\n      \"column\": \"max\",\\n      \"properties\": {\\n
        \"dtype\": \"date\",\\n        \"min\": 5.0,\\n        \"max\":
12382.5,\\n        \"num_unique_values\": 2,\\n        \"samples\": [\\n
5.0\\n        ],\\n        \"semantic_type\": \"\",\\n
\"description\": \"\\\"\\\"\\n      }\\n    }\\n  ]\\n}\", \"type\": \"dataframe\"}

```

```

# Identify top 10 products by 'number_of_reviews'

```

```

if 'number_of_reviews' in df.columns:
    top_10_products = df.sort_values(by='number_of_reviews',
ascending=False).head(10)
    print("\nTop 10 Products by Number of Reviews:")
    display(top_10_products[['title', 'number_of_reviews',
'numeric_rating']])
else:
    print("The 'number_of_reviews' column was not found in the
dataset.")

```

Top 10 Products by Number of Reviews:

```
{"summary": "{\n  \"name\": \"      print(\\\"\\\"\\\"The 'number_of_reviews'\n  column was not found in the dataset\\\",\\n  \"rows\": 10,\\n  \"fields\":\n  [\\n    {\\n      \"column\": \"title\\\",\\n      \"properties\": {\\n\n        \"dtype\": \"string\\\",\\n        \"num_unique_values\": 10,\\n\n        \"samples\": [\\n          \"HP 902 Black Ink Cartridge for HP Printers\n          | Works with Printer Series: OfficeJet 6950, 6960; OfficeJet Pro 6960,\n          6970 | Eligible for Instant Ink | T6L98AN\\\",\\n          \"Amazon\n          Basics 48-Pack AA Alkaline High-Performance Batteries, 1.5 Volt, 10-
          Year Shelf Life\\\",\\n          \"Apple iPhone XS Max, 64GB, Space Gray
          - Fully Unlocked (Renewed)\\\"\\n        ],\\n        \"semantic_type\":\n        \"\\\",\\n        \"description\": \"\\\"\\\"\\n      }\\n    },\\n    {\\n\n      \"column\": \"number_of_reviews\\\",\\n      \"properties\": {\\n\n        \"dtype\": \"Float64\\\",\\n        \"num_unique_values\": 1,\\n\n        \"samples\": [\\n          12382.5\\n        ],\\n\n        \"semantic_type\": \"\\\",\\n        \"description\": \"\\\"\\\"\\n      }\\n\n    },\\n    {\\n      \"column\": \"numeric_rating\\\",\\n\n      \"properties\": {\\n        \"dtype\": \"number\\\",\\n        \"std\":\n        0.24129281427805147,\\n        \"min\": 4.0,\\n        \"max\": 4.8,\\n\n        \"num_unique_values\": 6,\\n        \"samples\": [\\n          4.7\\n\n        ],\\n        \"semantic_type\": \"\\\",\\n        \"description\": \"\\\"\\\"\\n\n      }\\n    }\\n  ]\\n}\", \"type\": \"dataframe\"}
```

STEP 20: Save cleaned dataset

OUTPUT_FILE = "cleaned_dataset.csv"

```
df.to_csv(\n    OUTPUT_FILE,\n    index=False\n)
```

```
print(f"\\nCleaned dataset saved as: {OUTPUT_FILE}")
```

```
print("Final shape:", df.shape)
```

Cleaned dataset saved as: cleaned_dataset.csv

Final shape: (7855, 16)